Bootstrap Confidence Intervals

Implementation and Monte Carlo Evaluation

Silas Bambil and Yasmeen Babagana

Group: BB

STAT 8670: Computational Methods in Statistics
Georgia State University
Fall 2026

R package: `bootmean`, version 0.1.0

Submission version: hello world, this is our first work .

1 Introduction and project summary

A sample mean provides a point estimate of a population mean, but it does not show how much that estimate might change if new observations were collected. Confidence intervals address this uncertainty. Their usefulness depends on both their probability of containing the true parameter and their width. A narrow interval is attractive, but it offers weak protection if it misses the true mean too often.

This project implements percentile bootstrap confidence intervals for a population mean in an installable R package, **bootmean**. The bootstrap uses the observed sample as an empirical population and repeatedly samples from it with replacement. The variation among the resulting means approximates sampling uncertainty. This approach is connected to the resampling framework introduced by Efron (1979).

Our research question is: **How do sample size and population skewness affect the coverage, width, and computing time of percentile bootstrap intervals compared with standard t-intervals?** The comparison is deliberately focused on one parameter and one bootstrap interval construction. It evaluates a computational method rather than proposing a new statistical procedure.

We examined normal and exponential populations at sample sizes of 30 and 100. Each setting used 1,000 independently generated datasets and 2,000 bootstrap resamples per dataset. The random seed was 1998 at the start of each setting. We assessed empirical coverage, average interval width, and median time per interval.

The recorded results show narrower bootstrap intervals in every setting, accompanied by lower coverage than the corresponding t-intervals. For normal data at sample size 100, both methods were close to the 95% target. For exponential data, both methods under-covered in the recorded runs, even at sample size 100. Larger samples reduced widths in both populations. Bootstrap calculations were substantially slower than the direct t-interval formula.

The package was documented, tested, installed, and checked. The final reported package check completed with zero errors, zero warnings, and zero notes using `--no-manual`. Both authors contributed equally and worked together throughout the project.

2 Percentile bootstrap method

Let $X_1, \ldots, X_n$ be independent and identically distributed observations from a population F, with finite mean μ and finite variance σ^2. We estimate the mean using

$$\hat{\mu} = \overline{X} = \frac{1}{n} \sum_{i=1}^{n} X_i.$$

The empirical distribution assigns probability $1/n$ to each observed record. Repeated numerical values are counted as separate records. Conditional on the original sample, bootstrap sample b consists of n independent draws from this empirical distribution:

$$X_{1b}^*, \ldots, X_{nb}^* \sim \widehat{F}_n, \qquad \overline{X}_b^* = \frac{1}{n} \sum_{i=1}^{n} X_{ib}^*.$$

We repeat this process for $b = 1, \ldots, B$. For confidence level $1 - \alpha$, the percentile interval is

$$I_{\text{boot}} = \left[Q_{\alpha/2}(\overline{X}^*), Q_{1-\alpha/2}(\overline{X}^*) \right].$$

Here, Q_p is the sample quantile at probability p of the B saved bootstrap means. A nominal 95% interval uses the 2.5th and 97.5th percentiles. Our implementation uses R's default type-7 quantile method, which interpolates between order statistics (R Core Team, n.d.). With $B = 2,000$, these cutoffs are estimated from a finite resampling distribution rather than an exact enumeration of every possible resample.

For an illustrative sample of 10, 20, 30, 40, and 50, the mean is 30. Using seed 1998 and 2,000 resamples, the interval was [18, 42]. These endpoints describe uncertainty about a population mean; they do not indicate that 95% of individual observations should fall in that range.

Sampling with replacement is essential. Sampling all n observations without replacement would only rearrange them, leaving the mean unchanged. The ordinary bootstrap also depends on the independence assumption. Resampling individual observations is generally inappropriate for time series or clustered data without modifying the sampling scheme.

3 Reference interval and evaluation measures

The standard t-interval provides a simple reference for the population mean:

$$I_t = \left[\overline{X} - t_{1-\alpha/2,n-1}\frac{s}{\sqrt{n}}, \overline{X} + t_{1-\alpha/2,n-1}\frac{s}{\sqrt{n}}\right], \qquad s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(X_i - \overline{X})^2.$$

For independent normal observations, this interval has exact coverage $1-\alpha$ under the model. For nonnormal observations, the t-reference is an approximation. The normal setting therefore supplies a known theoretical benchmark, while the exponential setting examines performance when exact normal-theory justification does not apply.

For dataset m, let $[L_m, U_m]$ be an interval and let μ be the known population mean. The coverage indicator and width are

$$C_m = \mathbf{1}\{L_m \le \mu \le U_m\}, \qquad W_m = U_m - L_m.$$

Across $M = 1,000$ datasets, the estimated coverage and average width are

$$\hat{c} = \frac{1}{M}\sum_{m=1}^{M} C_m, \qquad \overline{W} = \frac{1}{M}\sum_{m=1}^{M} W_m.$$

Coverage assesses reliability; width assesses precision. Neither measure should be interpreted alone. An interval can achieve high coverage by being unnecessarily wide, or appear precise because it excludes plausible parameter values too often.

Because coverage is estimated from a finite experiment, we report its approximate Monte Carlo standard error:

$$\mathrm{MCSE}(\hat{c}) = \sqrt{\frac{\hat{c}(1-\hat{c})}{M}}.$$

We use $\hat{c} \pm 1.96\,\mathrm{MCSE}$ as approximate uncertainty bounds, truncated to [0, 1]. These are uncertainty bounds for the **coverage estimate**, not additional confidence intervals for the population mean. Bootstrap quantile estimation also introduces randomness within each dataset; the reported coverage pertains to the implemented procedure with $B = 2,000$.

4 Algorithm, implementation, and validation

The package exposes `bootstrap_mean_ci(x, B = 2000, conf = 0.95)`. Its core algorithm follows six steps:

1. Validate the data, resampling count, and confidence level.
2. Allocate a numeric vector of length B for bootstrap means.
3. Draw n values from the original data with replacement.
4. Calculate and store the resampled mean; repeat steps 3–4 B times.
5. Calculate the required lower and upper quantiles.
6. Return the original mean, interval endpoints, and saved bootstrap means.

We wrote the resampling loop and interval construction. Supporting operations use R's vector storage, `sample()`, `mean()`, and `stats::quantile()`. No existing bootstrap interval function replaces our core implementation. Storing only the current resample and the B means uses approximately $O(n + B)$ working storage rather than storing every resampled dataset. Resampling and averaging have an approximate $O(Bn)$ cost; quantile calculation adds its own processing cost.

The input checks reject nonnumeric or multidimensional data, fewer than two observations, missing or infinite values, invalid resampling counts, and confidence levels outside (0, 1). The default $B = 2,000$ is a simulation choice, not a universal accuracy guarantee. Allowing smaller valid counts supports flexibility but does not imply that such counts provide reliable tail quantiles.

Table 1: Automated validation cases and successful checks.

Test	Evidence	Checks
Constant observations	Mean and endpoints equal 5; all resampled means equal 5	4
Invalid inputs	Missing values, fractional B, and invalid confidence rejected	3
Reference agreement	Means and endpoints match index-based resampling	3

The package tests passed all 10 checks. Reference agreement used seed 1998 and a numerical tolerance of 10^{-12}. Constant data supplied a known-answer edge case, while invalid inputs checked error behavior. The full package check verified installation, namespace loading, documentation, examples, tests, and vignette rebuilding. It reported zero errors, warnings, and notes. The command used `--no-manual`, so generation of a PDF reference manual was excluded.

5 Simulation design and reproducibility

We considered four settings formed by two populations and two sample sizes. The normal population had mean 2 and standard deviation 1. The exponential population had rate 1, hence mean 1 and variance 1. Both populations therefore had the same variance, while differing in shape and location. Translation of the data does not change either method's coverage or interval width, making shape a useful focus of this comparison.

Table 2: Simulation settings; all intervals have nominal confidence level 95%.

Population	Parameters	n	True mean	Datasets M	Resamples B
Normal	Mean 2; SD 1	30	2	1000	2000
Normal	Mean 2; SD 1	100	2	1000	2000
Exponential	Rate 1	30	1	1000	2000
Exponential	Rate 1	100	1	1000	2000

For each setting, we initialized seed 1998 once, then allowed the generator to advance. Each repetition generated a new dataset and constructed both intervals from that same dataset. The bootstrap used 2,000 conditional resamples; the t-interval used the direct formula. We recorded endpoints, coverage indicators, and widths for each method. Reusing the same seed at the start of different settings supports reproducibility but does not make the settings independent paired experiments.

Two repetition levels must be distinguished. The outer Monte Carlo loop generates M datasets to evaluate a procedure. The inner bootstrap loop generates B resamples to construct one interval. Increasing M reduces uncertainty in estimated performance; increasing B improves the numerical approximation of each bootstrap interval. Neither automatically corrects finite-sample undercoverage.

The saved script `experiments/run_simulations.R` reproduced all eight previously recorded method-by-setting results. Separate scripts regenerate figures and timing tables. Records include `sessionInfo()` and `RNGkind()`. The reported check environment was R 4.4.1 on Apple Silicon macOS 26.5.2; detailed dependency versions are retained in the saved session-information files.

This report reads the simulation table from the saved project CSV. Knitting formats existing results; it does not rerun the simulation. The exact repository commit or release tag must be recorded before submission so the report, package, and presentation refer to one version.

6 Coverage results

Table 3: Estimated coverage and approximate Monte Carlo bounds; pp denotes percentage points.

Population	n	Method	Coverage	MCSE (pp)	MC bounds
Normal	30	Bootstrap	92.8%	0.82	91.2% to 94.4%
Normal	30	Standard t	94.8%	0.70	93.4% to 96.2%
Normal	100	Bootstrap	95.5%	0.66	94.2% to 96.8%
Normal	100	Standard t	95.7%	0.64	94.4% to 97.0%
Exponential	30	Bootstrap	91.4%	0.89	89.7% to 93.1%
Exponential	30	Standard t	92.5%	0.83	90.9% to 94.1%
Exponential	100	Bootstrap	91.4%	0.89	89.7% to 93.1%
Exponential	100	Standard t	92.2%	0.85	90.5% to 93.9%

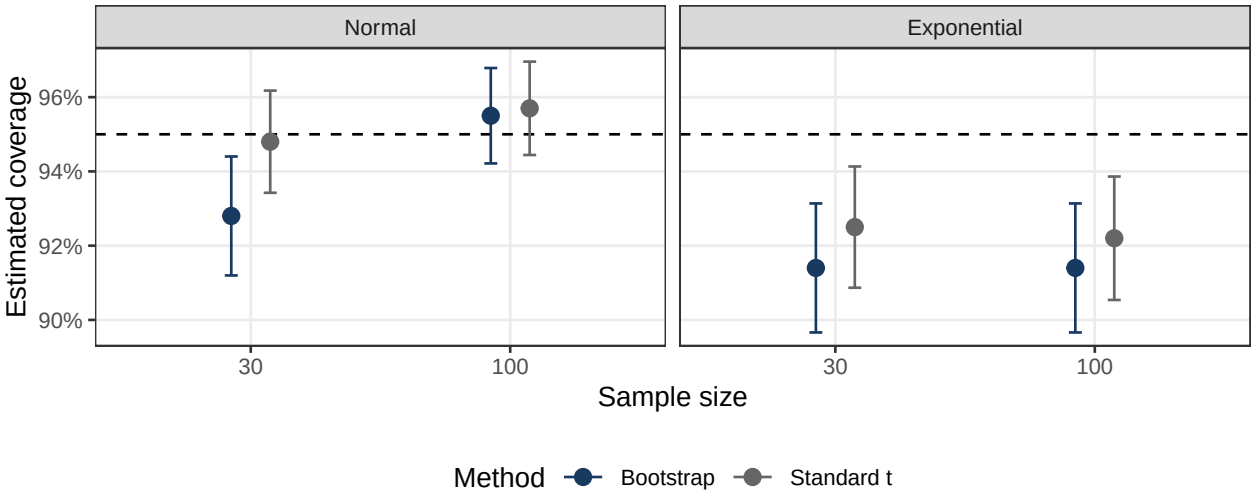


Figure 1: Coverage across 1,000 datasets per setting. Error bars are approximate 95% Monte Carlo bounds; the dashed line is the nominal 95% target.

For normal data with $n = 30$, bootstrap coverage was 92.8%, compared with 94.8% for the t-interval. At $n = 100$, coverage was 95.5% and 95.7%, respectively. Both normal-data bounds at $n = 100$ included 95%, consistent with the target given simulation uncertainty.

Exponential-data coverage remained below target: bootstrap coverage was 91.4% at $n = 30$ and 91.4% at $n = 100$; corresponding t-interval values were 92.5% and 92.2%. All exponential-data bounds excluded 95%. These are recorded finite-simulation findings, not a claim that coverage can never improve with increasing sample size. Overlapping error bars between methods are not a formal paired comparison.

7 Interval width and the precision–coverage tradeoff

Table 4: Average interval widths and relative reduction for bootstrap versus t-intervals.

Population	n	Bootstrap	Standard t	Bootstrap reduction (%)
Normal	30	0.6925	0.7373	6.0736
Normal	100	0.3877	0.3954	1.9519
Exponential	30	0.6733	0.7205	6.5592
Exponential	100	0.3820	0.3903	2.1287

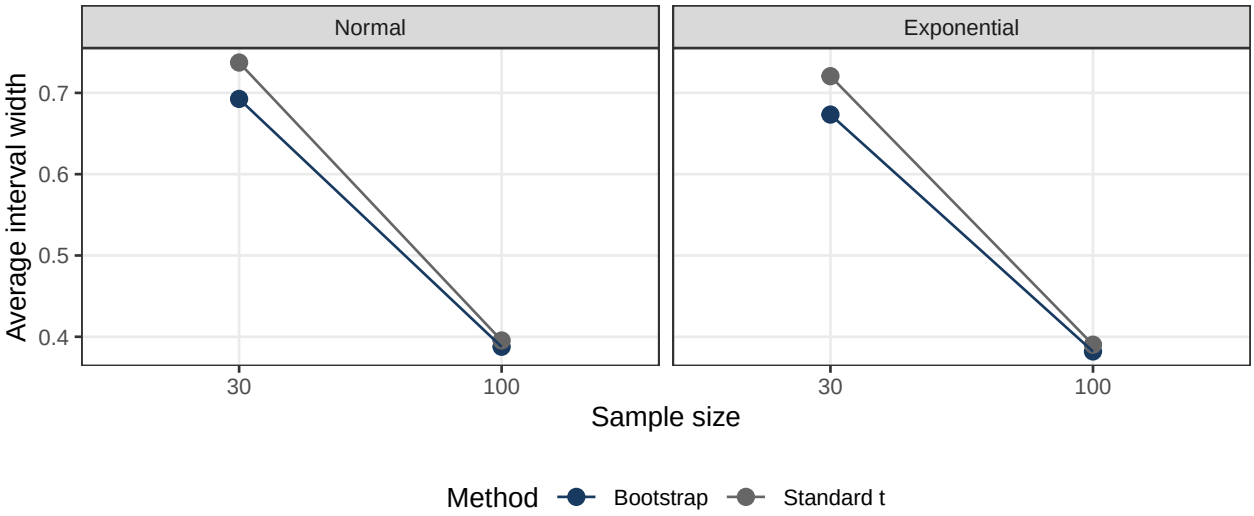


Figure 2: Average interval width over 1,000 datasets. Connecting lines summarize the two tested sample sizes; they do not describe untested intermediate sizes.

Both methods produced substantially narrower intervals at $n = 100$ than at $n = 30$. Bootstrap intervals were narrower in all four original settings, with smaller relative differences at $n = 100$. Two sample sizes are insufficient to establish a convergence rate.

Narrower intervals do not necessarily provide better inference. In the original experiment, bootstrap coverage was lower than t-interval coverage. The larger exponential validation, discussed next, found no established coverage advantage for either method.

Both populations had variance 1, making their interval widths comparable. These findings concern the mean under the selected distributions and should not be generalized automatically to other parameters.

8 Computing time

Timing experiments evaluated both methods on the same fixed dataset within each setting. We generated four datasets after setting seed 1998, then used `microbenchmark` for 50 calls per method. Data generation was outside the timed expressions. Bootstrap calls included 2,000 resamples and the construction of the returned result; t-interval calls returned the two endpoints.

Table 5: Median execution time per interval over 50 calls on one dataset per setting.

Setting	Method	Median time (ms)
Normal_30	Bootstrap	9.070122
Normal_30	Standard t	0.010619
Normal_100	Bootstrap	11.847708
Normal_100	Standard t	0.010373
Exponential_30	Bootstrap	8.766702
Exponential_30	Standard t	0.010230
Exponential_100	Bootstrap	11.630675
Exponential_100	Standard t	0.008712

The saved timing results show approximately 9 milliseconds for bootstrap at $n = 30$ and 12 milliseconds at $n = 100$. The t-interval took approximately 0.009–0.011 milliseconds. Bootstrap was substantially slower in every setting because it repeatedly samples and averages observations rather than applying a single formula.

The increase in bootstrap time with sample size is consistent with processing more observations in every resample. At a fixed sample size, times were similar across population shapes. The data values influence arithmetic results, but the resampling loop follows the same computational structure in both populations.

A warning reported reduced accuracy of nanosecond measurements to avoid potential integer overflow. We retain that warning as a limitation of the timing evidence. Median times provide a descriptive comparison, but differences among the very small t-interval timings should not be interpreted as meaningful distribution effects. Exact timing ratios also depend on hardware, system load, package versions, and implementation details.

These timings concern one fixed dataset per setting rather than average execution time across all 1,000 simulation datasets. In addition, bootstrap returns a vector of resampled means, whereas the t-function returns endpoints only. The benchmark consequently describes the implemented user-facing calls, not a perfectly output-matched comparison of abstract algorithms.

This report reads timing measurements from the saved project CSV. Repeating the timing script is expected to change the numerical values even when the data-generation seed is unchanged. Statistical results and timing reproducibility should therefore be distinguished.

9 Discussion, limitations, and useful extensions

The experiments illustrate a tradeoff between interval width, coverage, and computational cost. Percentile bootstrap intervals were narrower in the original four settings, while standard t-intervals were substantially faster. Resampling flexibility did not ensure better coverage.

For normal observations, the t-interval supplies an exact theoretical benchmark. Its empirical coverage was consistent with 95% within Monte Carlo uncertainty. Bootstrap coverage was below target at $n = 30$ and closer to target at $n = 100$.

9.1 Additional exponential validation

The original exponential results motivated an additional study using 10,000 samples of size 100, seed 1998, and 2,000 bootstrap resamples per sample. Both methods used the same datasets. This study supplements the original 1,000-simulation experiment.

Method	Coverage	Average width	95% Monte Carlo limits
Bootstrap	0.9402	0.3852244	[0.9356, 0.9448]
Standard t	0.9414	0.3933659	[0.9368, 0.9460]

Both coverage estimates were approximately 94%, and their Monte Carlo limits excluded the nominal 95% target. Bootstrap intervals were approximately 2.1% narrower on average.

The paired coverage difference, standard t minus bootstrap, was 0.0012. Its Monte Carlo standard error was 0.0009695, giving approximate 95% limits of [-0.00070024, 0.00310024]. These limits include zero, so the experiment does not establish a coverage advantage for either method.

Our package's interval endpoints also matched those calculated from the boot package's resampled means for one exponential sample, using the same seed, sequential resampling, and type-7 quantiles. This supports implementation correctness for that check; coverage is evaluated separately through simulation.

9.2 Limitations and extensions

The larger study provides a more precise estimate for exponential data at $n = 100$, but does not settle performance in the other settings. Only two population families and two sample sizes were considered. Monte Carlo limits used normal approximations, and uncertainty in average widths was not quantified.

Right skewness provides a plausible explanation for finite-sample undercoverage, but these experiments do not isolate its cause. Timing used one dataset per setting and was subject to timer precision and system variability.

Useful extensions include larger matched simulations for the remaining settings, varying the number of bootstrap resamples, and comparisons with bootstrap-t or bias-corrected and accelerated intervals.

10 Conclusion, reproduction, and contributions

We implemented percentile bootstrap intervals for a population mean and evaluated them against standard t-intervals. The package performs the main resampling and interval-construction steps directly, detects invalid inputs, returns interpretable outputs, and includes a runnable vignette and automated tests.

Larger samples reduced average interval widths in both populations. Bootstrap intervals were narrower in the original four settings, but substantially more expensive to compute. Both methods were close to 95% coverage for normal data at $n = 100$. An additional 10,000-simulation exponential study at $n = 100$ found approximately 94% coverage for both methods, with bootstrap intervals about 2.1% narrower. The paired comparison did not establish a coverage advantage for either method.

The evidence supports a qualified conclusion: percentile bootstrap intervals are straightforward to implement and use, but narrower intervals do not guarantee better inference. Users should assess coverage and assumptions alongside precision and computation. Our results do not establish that any one method is uniformly preferable across statistical problems.

10.1 Reproduce the project

After installing bootmean, run the following commands with the correct package path.

```
package_path <- "path/to/bootmean"
testthat::test_local(package_path)
source(file.path(package_path, "experiments", "run_simulations.R"), chdir = TRUE)
source(file.path(package_path, "experiments", "make_plots.R"), chdir = TRUE)
source(file.path(package_path, "experiments", "run_timing.R"), chdir = TRUE)
source(file.path(package_path, "experiments", "validate_exp100.R"), chdir = TRUE)
```

Results are saved in `experiments/results/` and figures in `experiments/figures/`. The README documents installation and checking. Include the retained check logs and keep all submission files consistent.

10.2 Contributions and assistance

Table 6: Group contributions, as reported by the authors.

Member	Contribution
Silas Bambil	Equal joint contribution to methods, code, experiments, validation, and reporting.
Yasmeen Babagana	Equal joint contribution to methods, code, experiments, validation, and reporting.

ChatGPT assisted with code, explanations, troubleshooting, and report drafting. The authors ran the analyses and remain responsible for the findings and compliance with course policy.

References

Efron, B. (1979). Bootstrap methods: Another look at the jackknife. *The Annals of Statistics*, **7**(1), 1–26. https://doi.org/10.1214/aos/1176344552

R Core Team. (n.d.). *Sample quantiles: Documentation for stats::quantile*. R documentation. https://stat.ethz.ch/R-manual/R-patched/library/stats/html/quantile.html

Kuang, C. (2026). *STAT 8670: Computational Methods in Statistics*. Course notes and project requirements, Georgia State University.

- Introduction to R: https://chikuang.github.io/teaching/2026F_stat8670/01-R.html
- Intro to computational statistics: https://chikuang.github.io/teaching/2026F_stat8670/02-computation.html
- Optimization: https://chikuang.github.io/teaching/2026F_stat8670/03-optimization.html
- Generating random variables: https://chikuang.github.io/teaching/2026F_stat8670/04-generating-rv.html
- Report requirements: https://chikuang.github.io/teaching/2026F_stat8670/Project-Report.html
- Package requirements: https://chikuang.github.io/teaching/2026F_stat8670/Project-Package.html

Software attribution: The package uses R and stats; tests use testthat; documentation uses roxygen2; the vignette and report use knitr and rmarkdown; graphics use ggplot2; timing uses microbenchmark. Exact versions should be taken from the retained session-information files rather than inferred from this report.